Arrays und Methoden

Einführung in die Programmierung 1
TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Mehrdimensionale Arrays in Java
- Kopieren von komplexeren Strukturen

Was wird in diesem Foliensatz besprochen?

- Arrays und Methoden
- Rekursion mit Arrays

Was lernen Sie kennen?

- Arrays als Parameter bei Methoden
- Arrays bei der Rückgabe
- Rekursionen auf Arrays

Arrays als Parameter

Übergabe

- Es wird die **Referenz** kopiert

Auswirkung

- **Änderungen am Inhalt** sind **außerhalb** der Methode sichtbar
- Änderungen am formalen Parameter selbst sind **nicht** außerhalb der Methode sichtbar

Beispiel (Array als Parameter)

```
public class CallTest {

    private static void test(int[] a) {
        int[] x = new int[3];
        a[1] = 3;
        a = x;
    }

    public static void main(String[] args) {
        int[] t = {5, 6, 7};
        System.out.println(t[1]);
        test(t);
        System.out.println(t[1]);
    }

}
```

Parameter a zeigt auf den gleichen Speicherbereich wie t

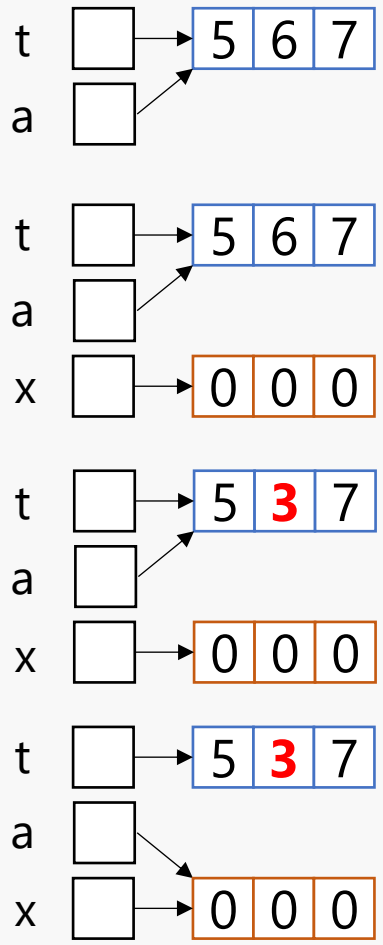
Diese Änderung ist auch in main sichtbar

Diese Zuweisung hat keine Auswirkung nach außen

```
6
3
```

Beispiel (Array als Parameter, Visualisierung)

```java
private static void test(int[] a) {

    int[] x = new int[3];

    a[1] = 3;

    a = x;
}
```



Arrays und return

```
private static int[] m() {
    …
    int[] a = new int[10];
    …
    return a;
}
```

Hier wird am Ende der Methode eine Referenz zurückgegeben

Muss außerhalb der Methode einer Array-Variable zugewiesen werden

- Mit dem Beenden der Methode wird die lokale Variable gelöscht
- Nach der Zuweisung gibt es aber eine Referenz auf das Array von außen

Beispiel (ArrayInverter)

Aufgabe

- Methode readArray
  - Liest die Anzahl der Elemente im Array ein
  - Liest Elemente von der Kommandozeile ein und gibt das Array zurück
- Methode printArray
  - Gibt die Elemente eines Arrays, das als Parameter übergeben wird, aus
- Methode invertArray
  - Dreht den Inhalt eines Arrays, das als Parameter übergeben wird, um
  - Es wird kein neues Array erzeugt (Seiteneffekt!)

Code in TUWEL

- ArrayInverter.java

Beispiel (MatrixMultiply)

Aufgabe

- Multiplikation zweier Matrizen (2-dimensionale Arrays)
- Ergebnis als Matrix (2-dimensionales Array) zurückgeben
- Rückgabe von null-Referenz
  - Null-Werte als Argumente oder als Array-Einträge
  - Matrizen lassen sich nicht multiplizieren (Dimensionen passen nicht)

Code in TUWEL

- MatrixMultiply.java

Kommandozeilenargumente

- main-Methode hat als Parameter ein Array von Strings
- Argumente übergeben
  - Kommandozeile: Nach dem Aufruf des Programms

    Beispiel: java ParameterTest **Hello 1 2**
  - IntelliJ: Run -> Edit Configurations -> Programm arguments
- Ausgabe der Argumente

```
public static void main(String[] args) {
    for (String s : args) {
        System.out.println(s);
    }
    …
}
```

```
Hello
1
2
```

Kommandozeilenargumente verarbeiten

```
int grade = Integer.parseInt(args[0]);
String text = " no grade ";
if (grade == 1) {
    text = " excellent ";
} else if (grade == 2) {
    text = " good ";
} else if (grade == 3) {
    text = " satisfactory ";
} else if (grade == 4) {
    text = " sufficient ";
} else if (grade == 5) {
    text = " insufficient ";
}
System.out.println("Your grade:" + text);
```

args[0] entspricht dem ersten Kommandozeilenargument.
Allgemein: Wird nichts (oder zu wenige Argumente) übergeben, dann gibt es eine Ausnahme.

Methode parseInt in der Klasse Integer versucht einen String in einen Integer umzuwandeln

Methoden mit variabler Parameteranzahl

Herkömmliche Lösung mit einem Array

```
private static int sum(int[] values) { … }
```

varargs-Parameter

```
private static int sum(int… values){ … }
```

- Aus dem varargs-Parameter wird automatisch ein Array erzeugt
- Der varargs-Parameter muss immer an der letzten Stelle der Parameterliste stehen

Beispiel für Verwendung von varargs in TUWEL - `VarargsTest.java`

Arrays und Rekursion

Bei rekursiven Methoden muss genau auf den Umgang mit Referenzen geachtet werden!

Beispiel (rekursiv Array-Inhalte ausgeben)

```java
public class RecursiveArrayOutputTest {

    // arr.length > 0 && 0 <= left < arr.length && 0 <= right < arr.length
    private static void printArrayRecursively(int[] array, int left, int right) {
        if (left <= right) {
            System.out.print(array[left] + " ");
            printArrayRecursively(array, left + 1, right);
        }
    }

    public static void main(String[] args) {
        int[] x = new int[]{1, 5, 4, 2, 6, 8, 0, 7};
        printArrayRecursively(x, 0, x.length - 1);
        System.out.println();
        printArrayRecursively(x, 2, 5);
    }

}
```

Basisfall (keine Ausgabe) ist hier implizit

```
1 5 4 2 6 8 0 7
4 2 6 8
```

Beispiel (maximale Differenz finden)

```java
public class RecursiveMaximumDifference {

    // array != null, array.length >= 1, 0 <= i < array.length
    private static int findMaxDiff(int[] array, int i) {
        if (i > 0) {
            int localDiff = Math.abs(array[i - 1] - array[i]);
            int diff = findMaxDiff(array, i - 1);
            return Math.max(localDiff, diff);
        }
        return 0;
    }

    public static void main(String[] args) {
        int[] arr = new int[]{3, 7, 1, 8, 2, 4, 0, 9};
        System.out.println(findMaxDiff(arr, arr.length - 1));
        System.out.println(findMaxDiff(arr, arr.length/2));
        System.out.println(findMaxDiff(arr, arr.length/4));
        System.out.println(findMaxDiff(new int[]{20}, 0));
    }

}
```

Diese Methode berechnet die maximale absolute Differenz zweier aufeinander folgender Elemente von array im Indexbereich 0 bis i (inklusive).

```
9
7
6
0
```

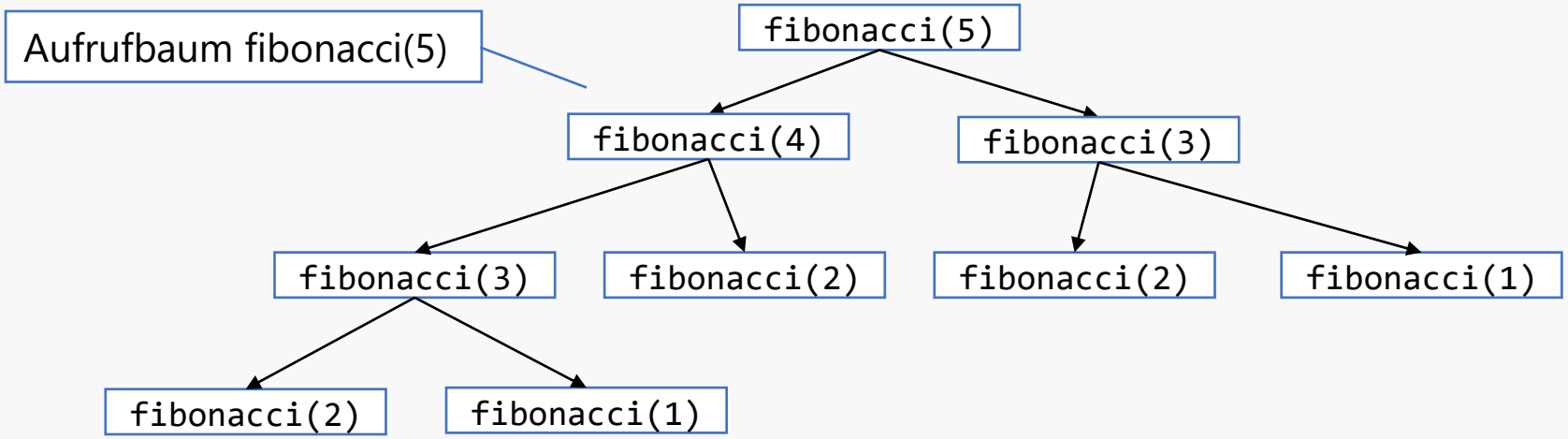
Memoisation

Rückgabewerte von Funktionen werden zwischengespeichert

Keine neue Berechnung, wenn Aufrufergebnis schon gespeichert

Fibonacci-Zahlen (Wiederholung)

```
private static long fibonacci(int n) {
    if (n <= 2) {
        return 1;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```



Beispiel (Memoisation)

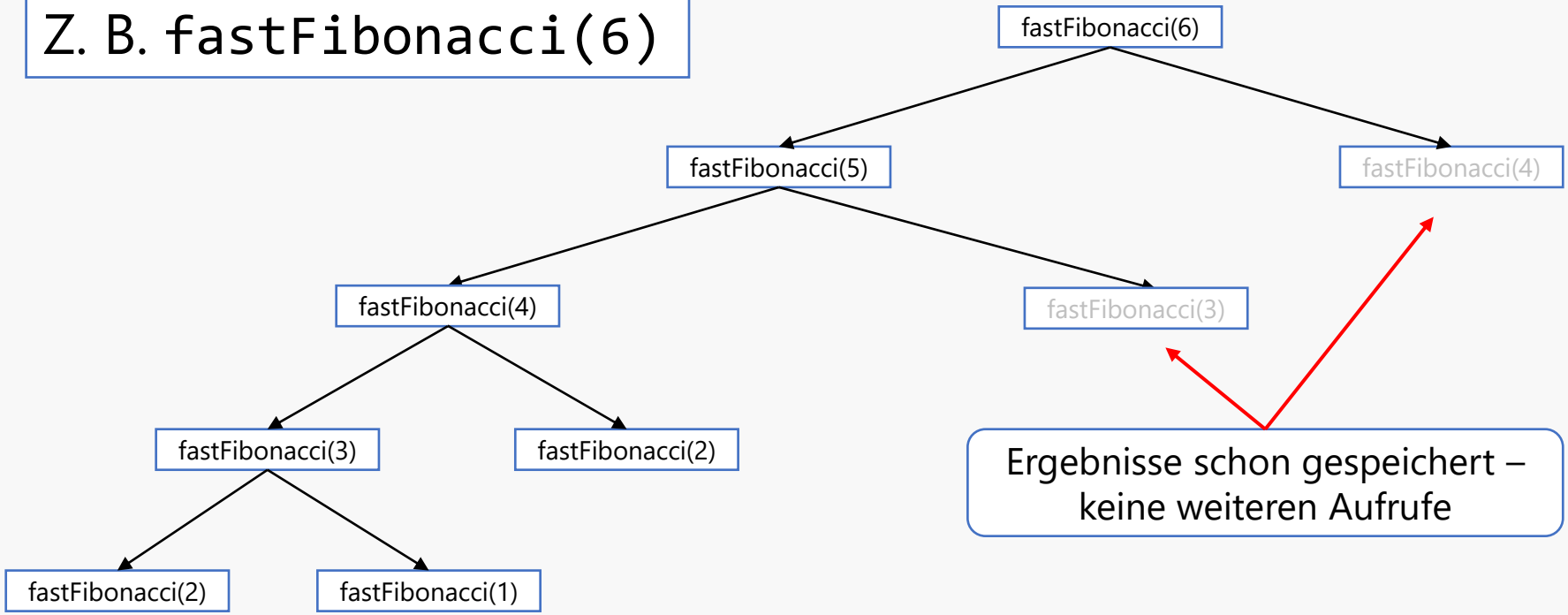
```java
public class FibonacciWithMemoization {

    private static final long[] f = new long[93];

    // 1 <= n <= 92
    private static long fastFibonacci(int n) {
        if (n <= 2) {
            return 1;
        } else if (f[n] == 0) {
            f[n] = fastFibonacci(n - 1) + fastFibonacci(n - 2);
        }
        return f[n];
    }

    public static void main(String[] args) {
        System.out.println(fastFibonacci(50));
        System.out.println(fastFibonacci(90));
        System.out.println(fastFibonacci(80));
    }

}
```

Speicher für Ergebnisse - nur bis 92 sinnvoll, danach arithmetischer Überlauf

```
12586269025
2880067194370816120
23416728348467685
```

Memoisation (Anzahl der Aufrufe)

Z. B. `fastFibonacci(6)`



Memoisation (Anzahl der Aufrufe)

Wie viele Aufrufe finden bei unterschiedlichen Werten für *n* (abhängig von den vorherigen Aufrufen) statt?

```
…
    public static void main(String[] args) {
        System.out.println(fastFibonacci(50));
        System.out.println(fastFibonacci(90));
        System.out.println(fastFibonacci(80));
    }
…
```

```
fastFibonacci(50) -> 97 Aufrufe
fastFibonacci(90) -> 81 Aufrufe
 fastFibonacci(80) -> 1 Aufruf
```